

I. METHODOLOGY

This section describes the end-to-end pipeline adopted for explainable Bangla document classification, including dataset acquisition, preprocessing, label encoding, sequence representation, the three neural architectures compared in this study (CNN, BiLSTM, and Attention-BiLSTM), the training protocol, the evaluation metrics, and the gradient-based explainability mechanism used to interpret model predictions.

A. Dataset

In this study, the corpus that is used is a large scale Bangla newspaper corpus. Made up of news articles saved in JSON format, with each record being a single article. The document includes an title, content and a matching category label. The title and content columns are added for each article. They were then all put together in one text sample. This would be a huge undertaking because of its size. the original corpus, a representative subset of 100,000 samples was A sample of drawings created for model development and evaluation. This produced the following set of data: was presented in a tabular format, containing two main fields: the raw text and the label for its category. The class distribution by categories was visualized to evaluate the balance of data sets, which was then followed by. To explain the selection of assessment indicators (macro/weighted) The precision, recall, and F1-score are calculated to take into consideration potential class imbalance. Fig. 1 shows the distribution of samples across categories.

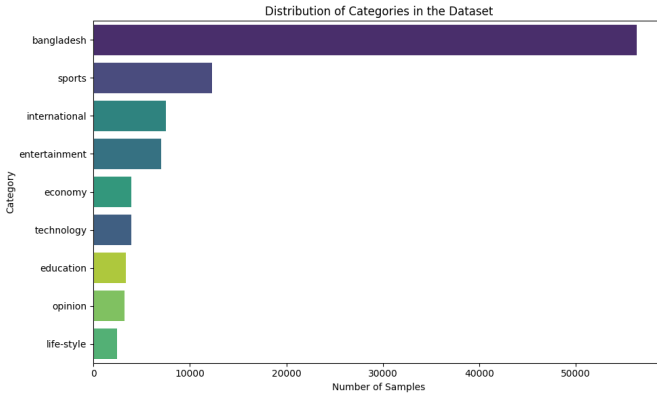


Fig. 1. Distribution of categories in the Bangla newspaper dataset.

B. Text Preprocessing

Raw Bangla news text is noisy in nature, with URLs, non-Bengali characters and many other things. Characters, punctuation artifacts and irregular whitespace. To normalize the input to the tokenizer and the network to be used downstream, It was used to clean all text samples, and the cleaning function was applied to each of them. following sequential operations:

- 1) **URL removal:** Strings matching the patterns `http\S+` and `www\S+` were removed, as hyperlinks carry no semantic value for classification and introduce vocabulary noise.

- 2) **Non-Bengali character filtering:** All characters outside the Unicode range `U+0980--U+09FF` (the Bengali script block) were replaced with whitespace, ensuring the model is exposed only to native Bengali graphemes and is not confounded by Latin characters, digits, or emojis.
- 3) **Whitespace normalization:** Consecutive whitespace characters were collapsed into a single space, and leading/trailing whitespace was stripped.

This cleaning process maintains the corpus' linguistic content with the elimination of artifacts which would otherwise overestimate vocabulary size. and confuse the semantic information presented to the embedding layer.

C. Label Encoding

The categorical class labels (e.g., news categories, like politics, ...) A series of integer indices were used to represent the sport and entertainment types of questions. Label encoder is used for creating a bijective mapping between each category name. and an integer identifier that is unique. This is an integer encoded label that was used as the target variable for the sparse categorical cross-entropy loss Parameters used in model training.

D. Train-Test Split

The cleaned and labeled data set was divided into training and test set. Subsets with stratified split of which 85% were assigned to. One-fifth of these, or 15% were withheld for testing. Stratification with regard to the label distribution guarantees that the relative proportion of each category is balanced between both subsets, thereby reducing the bias in the Class imbalance effect on evaluation.

E. Tokenization and Sequence Padding

All text samples were translated to fixed length numeric sequences to be These were fed into the neural networks. A word-level tokenizer was fitted. A word-level tokenizer was installed. only on the vocabulary used in the training corpus, limiting the vocabulary to the 50,000 most frequent tokens (`MAX_WORDS = 50,000`) and Mapping any out-of-vocabulary word to a token that is reserved for it, that is, `<OOV>`. To each text sample, the corresponding sequence of integer token was then obtained. The sequence of indices and padded (or truncated) to a fixed length of 400 tokens Using post-padding and post-truncation with (`MAX_LEN = 400`), we see that the weights are: The weights are: that all the input tensors have a common shape for batch processing.

F. Model Architectures

Three architectures of the neural network of increasing representational complexity. were trained and designed under the same conditions of data to allow a controlled comparison: a Convolutional Neural Network (CNN), LSTM network. Bidirectional Long Short-Term Memory network (BiLSTM), and an Attention-augmented BiLSTM (Attention-BiLSTM). All three architectures have a shared embedding

layer which assigns every token index to a dense A 128-dimensional vector in which the values are learned during training.

1) *CNN Model*: The CNN architecture uses a 1-dimensional convolution The results were obtained using the Conv1D module containing a set of 128 filters and a kernel size of 5 over the embedded token sequence, followed by a ReLU activation to learn local n -grams of patterns that represent specific categories. A global Then max-pooling operation takes the variable-length feature maps to a fixed length of A fixed-size vector that will store the most prominent activation (eigenvector) for each filter. This is followed by a dense layer with 64 units and ReLU activation function. A dropout layer (rate = 0.5) to perform regularization and a are added between the activation and the The last softmax output layer has the same number of units as there are classes. The CNN is used as a lightweight baseline that reflects local lexical information.CNN is used as a lightweight baseline capturing local lexical information. efficiently. The layer-wise structure of the CNN model is shown in Fig. 2. of the CNN model.

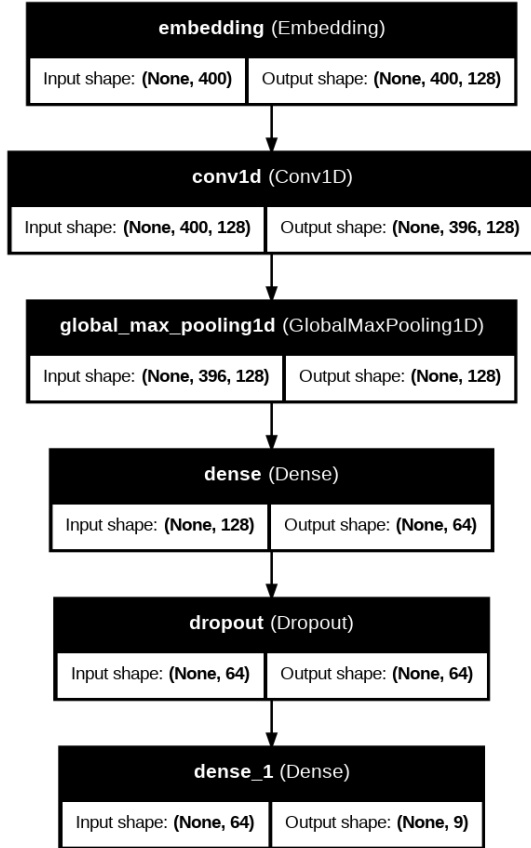


Fig. 2. Layer-wise architecture of the CNN model.

2) *BiLSTM Model*: The BiLSTM architecture uses a different feature extractor, the convolutional one is replaced by a normal LSTM. A bidirectional LSTM with 64 hidden units in both directions (dropout = 0.3), both forward and backward. use backward directions to learn long-range contextual rela-

tionships They play a key role in Bangla sentence structure. The final The same dense–dropout–softmax is used for hidden representation. head used in the CNN model. Fig. 3 illustrates the BiLSTM model structure (layer-wise).

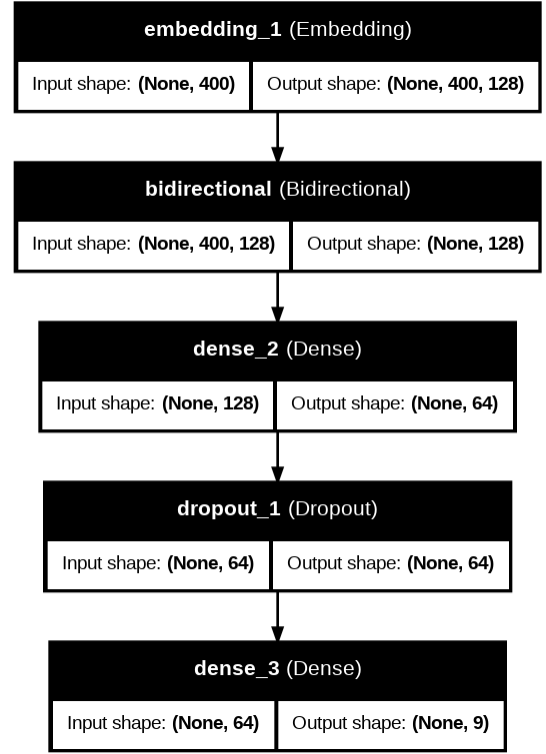


Fig. 3. Layer-wise architecture of the BiLSTM model.

3) *Attention-BiLSTM Model*: The proposed Attention-BiLSTM architecture extends the BiLSTM by retaining the full sequence of hidden states (`return_sequences = True`) rather than collapsing them into a single vector. A scaled dot-product attention mechanism is applied over the BiLSTM hidden states, allowing the model to assign a learned importance weight α_t to each time step t :

$$\alpha_t = \frac{\exp(\text{score}(h_t, h_t))}{\sum_{k=1}^T \exp(\text{score}(h_k, h_k))} \quad (1)$$

where h_t denotes the BiLSTM hidden state at time step t and T is the sequence length. The attention output is a weighted combination of the hidden states, followed by global average pooling to produce a fixed-size context vector. This vector is subsequently passed through the same dense (64 units, ReLU), dropout (0.5), and softmax output layers as the previous two models. By explicitly weighting the contribution of each token, the attention mechanism both improves classification performance on longer documents and provides an interpretable signal that is later exploited for explainability analysis. The complete layer-wise architecture of the Attention-BiLSTM model, including input/output tensor shapes at every stage, is shown in Fig. 4.

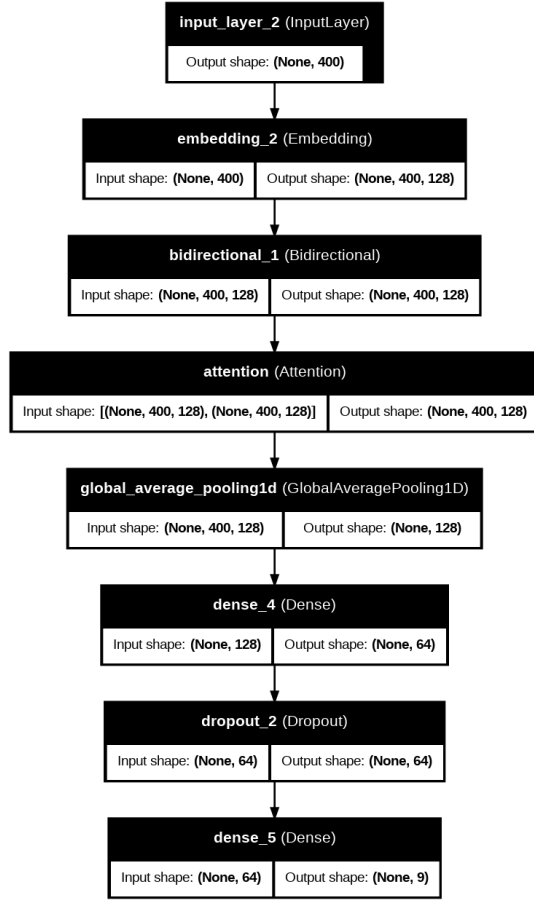


Fig. 4. Layer-wise architecture of the proposed Attention-BiLSTM model, generated via Keras' model visualization utility.

G. Training Protocol

The three models were put together using the Adam optimizer and the sparse categorical cross-entropy loss function. We looked at how accurate the classification was as the thing to focus on while training. The CNN and BiLSTM models were trained for 5 epochs. The CNN model used a batch size of 128. The BiLSTM model used a batch size of 64. We took 10 percent of the training set. Used it to check how the models were doing internally. The Attention-BiLSTM model was the model we were interested in. We trained this Attention-BiLSTM model for up to 15 epochs, with a batch size of 64. We also used some tools to help the Attention-BiLSTM model train better and to keep it stable.

- **Early Stopping:** We stopped training if the validation loss was not getting better for three epochs, in a row. We did this to prevent the model from overfitting. The model weights that worked best were kept so we could use them again..
- **Reduce Learning Rate on Plateau:** The learning rate was reduced by a factor of 0.5 if the validation loss plateaued for 2 consecutive epochs, allowing finer convergence in later training stages.
- **Model Checkpointing:** The best-performing model (low-

est validation loss) was persisted to disk during training to guarantee that the final evaluated model corresponds to the optimal checkpoint rather than the last training epoch.

H. Explainability Analysis

To make the proposed system easier to understand a new method was added to the Attention-BiLSTM model. This method looks at how each part of the input text affects the result. When the model gets some text it runs the text through all its layers like the embedding layer, the BiLSTM, the attention, the pooling, the dense and the output layers. The model keeps track of what it does at each step. Then it figures out how each word in the text affects the result. It does this by working from the result to the beginning, which is called backpropagation. The Attention-BiLSTM model uses this to understand how the words, in the text impact the predicted class score.

$$s_t = \left\| \frac{\partial \hat{y}_c}{\partial e_t} \odot e_t \right\| \quad (2)$$

Where $\hat{y}_c$ is the predicted probability of the class c , the predicted probability is very important. E_t is the embedding vector of t , and $\odot$ denotes element-wise multiplication. The resulting saliency score s_t for each token is then normalized across the sequence. This produces an importance score. It highlights the top contributing Bangla tokens for a given prediction. The saliency score s_t helps to understand which tokens are important. This gradient $\times$ input attribution approach provides post-hoc explanations. These explanations are at the instance level. They reveal which words most influenced the model's decision. The model debugging is supported by these explanations. The trust calibration is also supported. The qualitative validation is done to check if the model's decisions align with linguistic cues for each category. An example of the resulting token-level explanation, for a sample prediction is shown in Fig. 5. The normalized token importance scores highlight the Bangla words. These words strongly influenced the model's classification decision. The Bangla tokens are very important. The model's decision is based on these tokens.

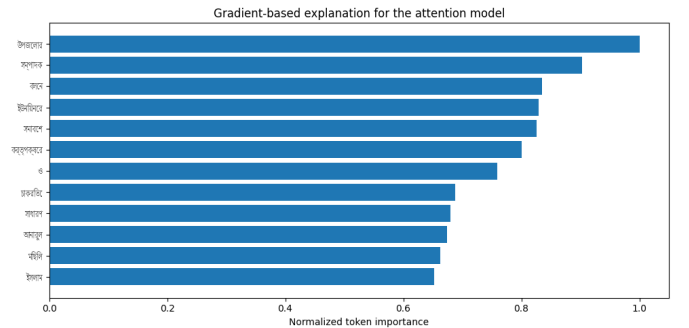


Fig. 5. Gradient-based token importance scores for a sample prediction from the Attention-BiLSTM model.